



Authenticating to the REST API

You can authenticate to the REST API to access more endpoints and have a higher rate limit.

[Copy as Markdown](#)

In this article

[About authentication](#)[Authenticating with a personal access token](#)[Authenticating with a token generated by an app](#)[Authenticating in a GitHub Actions workflow](#)[Authenticating with username and password](#)[Further reading](#)

About authentication

Many REST API endpoints require authentication or return additional information if you are authenticated. Additionally, you can make more requests per hour when you are authenticated.

To authenticate your request, you will need to provide an authentication token with the required scopes or permissions. There are a few different ways to get a token: You can create a personal access token, generate a token with a GitHub App, or use the built-in `GITHUB_TOKEN` in a GitHub Actions workflow.

After creating a token, you can authenticate your request by sending the token in the `Authorization` header of your request. For example, in the following request, replace `YOUR-TOKEN` with a reference to your token:

```
curl --request GET \  
--url "https://api.github.com/octocat" \  

```

```
--header "Authorization: Bearer YOUR-TOKEN" \  
--header "X-GitHub-API-Version: 2026-03-10"
```

Note

In most cases, you can use `Authorization: Bearer` or `Authorization: token` to pass a token. However, if you are passing a JSON web token (JWT), you must use `Authorization: Bearer .`

Failed login limit

If you try to use a REST API endpoint without a token or with a token that has insufficient permissions, you will receive a `404 Not Found` or `403 Forbidden` response. Authenticating with invalid credentials will initially return a `401 Unauthorized` response.

After detecting several requests with invalid credentials within a short period, the API will temporarily reject all authentication attempts for that user (including ones with valid credentials) with a `403 Forbidden` response. For more information, see [Rate limits for the REST API](#).

Authenticating with a personal access token

If you want to use the GitHub REST API for personal use, you can create a personal access token. If possible, GitHub recommends that you use a fine-grained personal access token instead of a personal access token (classic). For more information about creating a personal access token, see [Managing your personal access tokens](#).

If you are using a fine-grained personal access token, your fine-grained personal access token requires specific permissions in order to access each REST API endpoint. The REST API reference document for each endpoint states whether the endpoint works with fine-grained personal access tokens and states what permissions are required in order for the token to use the endpoint. Some endpoints may require multiple permissions, and some endpoints may require one of multiple permissions. For an overview of which REST API endpoints a fine-grained personal access token can access with each permission, see [Permissions required for fine-grained personal access tokens](#).

If you are using a personal access token (classic), it requires specific scopes in order to access each REST API endpoint. For general guidance about what scopes to choose, see [Scopes for OAuth apps](#).

Personal access tokens act as your identity (limited by the scopes or permissions you selected) when you make requests to the REST API. As such, it is important to keep your personal access tokens secure. For more information about keeping your personal access tokens secure, see [Keeping your API credentials secure](#).

Personal access tokens and SAML SSO

If you use a personal access token (classic) to access an organization that enforces SAML single sign-on (SSO) for authentication, you will need to authorize your token after creation. Fine-grained personal access tokens are authorized during token creation, before access to the organization is granted. For more information, see [Authorizing a personal access token for use with single sign-on](#).

If you do not authorize your personal access token (classic) for SAML SSO before you try to use it to access a single organization that enforces SAML SSO, you may receive a `404 Not Found` or a `403 Forbidden` error. If you receive a `403 Forbidden` error, the `X-GitHub-SSO` header will include a URL that you can follow to authorize your token. The URL expires after one hour.

If you do not authorize your personal access token (classic) for SAML SSO before you try to use it to access multiple organizations, the API will not return results from the organizations that require SAML SSO and the `X-GitHub-SSO` header will indicate the ID of the organizations that require SAML SSO authorization of your personal access token (classic). For example: `X-GitHub-SSO: partial-results; organizations=21955855,20582480`.

Authenticating with a token generated by an app

If you want to use the API for an organization or on behalf of another user, GitHub recommends that you use a GitHub App. For more information, see [About authentication with a GitHub App](#).

The REST API reference documentation for each endpoint states whether the endpoint works with GitHub Apps and states what permissions are required in order for the app to use the endpoint. Some endpoints may require multiple permissions, and some endpoints may require one of multiple permissions. For an overview of which REST API endpoints a GitHub App can access with each permission, see [Permissions required for GitHub Apps](#).

You can also create an OAuth token with an OAuth app to access the REST API. However, GitHub recommends that you use a GitHub App instead. GitHub Apps allow more control over the access and permission that the app has.

Access tokens created by apps are automatically authorized for SAML SSO.

Using basic authentication

Some REST API endpoints for GitHub Apps and OAuth apps require you to use basic authentication to access the endpoint. You will use the app's client ID as the username and the app's client secret as the password.

For example:

```
curl --request POST \
--url "https://api.github.com/applications/YOUR_CLIENT_ID/token" \
--user "YOUR_CLIENT_ID:YOUR_CLIENT_SECRET" \
--header "Accept: application/vnd.github+json" \
--header "X-GitHub-Api-Version: 2026-03-10" \
--data '{
  "access_token": "ACCESS_TOKEN_TO_CHECK"
}'
```

The client ID and client secret are associated with the app, not with the owner of the app or a user who authorized the app. They are used to perform operations on behalf of the app, such as creating access tokens.

If you are the owner of a GitHub App or OAuth app, or if you are an app manager for a GitHub App, you can find the client ID and generate a client secret on the settings page for your app. To navigate to your app's settings page:

- 1 In the upper-right corner of any page on GitHub, click your profile picture.
- 2 Navigate to your account settings.
 - For an app owned by a personal account, click **Settings**.
 - For an app owned by an organization:
 - a. Click **Your organizations**.
 - b. To the right of the organization, click **Settings**.
- 3 In the left sidebar, click <> **Developer settings**.
- 4 In the left sidebar, click **GitHub Apps** or **OAuth apps**.
- 5 For GitHub Apps, to the right of the GitHub App you want to access, click **Edit**. For OAuth apps, click the app that you want to access.
- 6 Next to **Client ID**, you will see the client ID for your app.
- 7 Next to **Client secrets**, click **Generate a new client secret** to generate a client secret for your app.

Authenticating in a GitHub Actions workflow

If you want to use the API in a GitHub Actions workflow, GitHub recommends that you authenticate with the built-in `GITHUB_TOKEN` instead of creating a token. You can grant permissions to the `GITHUB_TOKEN` with the `permissions` key. For more information, see [Use GITHUB_TOKEN for authentication in workflows](https://docs.github.com/en/rest/authentication/authenticating-to-the-rest-api?apiVersion=2026-03-10).

If this is not possible, you can store your token as a secret and use the name of your secret in your GitHub Actions workflow. For more information about secrets, see [Using secrets in GitHub Actions](#).

Authenticating in a GitHub Actions workflow using GitHub CLI

To make an authenticated request to the API in a GitHub Actions workflow using GitHub CLI, you can store the value of `GITHUB_TOKEN` as an environment variable, and use the `run` keyword to execute the GitHub CLI `api` subcommand. For more information about the `run` keyword, see [Workflow syntax for GitHub Actions](#).

In the following example workflow, replace `PATH` with the path of the endpoint. For more information about the path, see [Getting started with the REST API](#).

```
jobs:
  use_api:
    runs-on: ubuntu-latest
    permissions: {}
    steps:
      - env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: |
          gh api /PATH
```

Authenticating in a GitHub Actions workflow using `curl`

To make an authenticated request to the API in a GitHub Actions workflow using `curl`, you can store the value of `GITHUB_TOKEN` as an environment variable, and use the `run` keyword to execute a `curl` request to the API. For more information about the `run` keyword, see [Workflow syntax for GitHub Actions](#).

In the following example workflow, replace `PATH` with the path of the endpoint. For more information about the path, see [Getting started with the REST API](#).

YAML



```
jobs:
  use_api:
    runs-on: ubuntu-latest
    permissions: {}
    steps:
      - env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: |
          curl --request GET \
```

```
--url "https://api.github.com/PATH" \  
--header "Authorization: Bearer $GH_TOKEN"
```

Authenticating in a GitHub Actions workflow using JavaScript

For an example of how to authenticate in a GitHub Actions workflow using JavaScript, see [Scripting with the REST API and JavaScript](#).

Authenticating with username and password

Authentication with username and password is not supported. If you try to authenticate with user name and password, you will receive a 4xx error.

Further reading

- [Keeping your API credentials secure](#)
- [Getting started with the REST API](#)

Legal

© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Status](#) [Pricing](#) [Expert services](#) [Blog](#)